



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Ingeniería en
Software

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Ingeniería en
Software



INFORME DE LABORATORIO N.º 2 · III UNIDAD

Solución distribuida con microservicios

*Diseño e implementación de un sistema de procesamiento
distribuido de imágenes con detección de rostros*

Asignatura: Computación Paralela · NRC 31485
Docente: Ing. Omar Rolando Quimbíta Chiluisa
Práctica N.º: 7 · III Unidad · Nivel: Cuarto
Período: Abril 2026 – Agosto 2026 · Fecha: 22/07/2026

INTEGRANTES DEL GRUPO

 Chiluisa Ponce, David Sebastián

 Medina Quinga, Justin Vladimir

 Untuña Valiente, Johan Jaren

 Yugcha Ledesma, Robert Xavier

Contenido

1. Introducción	1
2. Objetivos	2
3. Marco teórico	2
3.1. Arquitectura de microservicios	2
3.2. Comunicación HTTP/REST y serialización	3
3.3. Patrón pipeline y patrón productor–consumidor	3
3.4. Detección de rostros: Haar (Viola–Jones) + NMS	3
4. Actividad 1: Análisis del problema	4
4.1. Flujo general de procesamiento	4
4.2. Tareas independientes y tareas que se comunican	4
5. Actividad 2: Diseño de la arquitectura	5
5.1. Diagrama de microservicios	5
5.2. Entradas, salidas y responsabilidades	5
5.3. El “sobre” de datos (envelope)	5
6. Actividad 3: Implementación de microservicios	6
6.1. Utilidades compartidas (<code>comun.py</code>)	6
6.2. MS1 – Recepción	6
6.3. MS2 – Preprocesamiento	6
6.4. MS3 – Análisis (detección de rostros)	7
6.5. MS4 – Almacenamiento	7
7. Actividad 4: Comunicación distribuida	7
7.1. Mecanismo de intercambio	7
7.2. Validación del paso de datos	7
7.3. Tolerancia a fallos: reintentos con backoff	7
8. Actividad 5: Pruebas del sistema	8
8.1. Evidencia de imágenes procesadas	8
8.2. Tiempos por etapa	9
8.3. Secuencial vs. concurrente	9
8.4. Secuencia de ejecución observada	9
9. Actividad 6: Análisis desde la computación paralela	9
9.1. ¿Qué partes se ejecutan en paralelo?	9
9.2. Cuello de botella y ley de Amdahl	10
9.3. Cómo mejora la arquitectura la organización y escalabilidad	10
10. Resultados obtenidos	10
11. Conclusiones	10
12. Recomendaciones	11

1 Introducción

El procesamiento de imágenes se resuelve tradicionalmente con una *única* aplicación monolítica encargada de recibir la imagen, procesarla, detectar los elementos de interés y almacenar el resultado. Ese enfoque, aunque sencillo, dificulta el mantenimiento, limita la escalabilidad y acopla etapas que en realidad son independientes.

En este laboratorio, el grupo diseña e implementa una **solución distribuida basada en microservicios**: el sistema completo se divide en servicios pequeños, autónomos y especializados, donde cada uno ejecuta una tarea concreta del flujo de procesamiento visual y se comunica con los demás mediante solicitudes **HTTP**. Concretamente se construye un *pipeline* de cuatro etapas —recepción, preprocesamiento, análisis (detección de rostros) y almacenamiento— que se ejecutan como procesos independientes.

Desde la perspectiva de la **computación paralela**, esta arquitectura es relevante porque varias etapas pueden ejecutarse al mismo tiempo: mientras un microservicio recibe una imagen, otro puede estar preprocesando la anterior y un tercero detectando rostros en otra. Así, la práctica integra en una sola experiencia los conceptos de **paralelismo**, **sistemas distribuidos** y **visión por computadora**.

En una mirada

- 4 **microservicios** independientes (Flask) comunicados por HTTP/JSON.
- Detección de rostros con **Haar (Viola–Jones)** + NMS sobre OpenCV.
- Se procesaron **6 imágenes** y se detectaron **34 rostros**.
- El modo concurrente logró un **speedup de $2.47\times$** (59.5 % de mejora).

2 Objetivos

Objetivo general

Diseñar e implementar una solución distribuida con microservicios para el procesamiento de imágenes, aplicando principios de computación paralela para organizar, distribuir y evaluar el flujo de trabajo del sistema.

Objetivos específicos

- Analizar el flujo de procesamiento de imágenes y separar sus tareas en componentes independientes.
- Diseñar una arquitectura de microservicios con responsabilidades bien definidas y un mecanismo de comunicación entre ellos.
- Implementar al menos cuatro microservicios funcionales que cooperen en un *pipeline*.
- Establecer la comunicación distribuida mediante HTTP y validar el paso correcto de la información entre etapas.
- Ejecutar el sistema con varias imágenes y medir tiempos de respuesta y comportamiento.
- Relacionar la arquitectura con los patrones *pipeline* y *productor–consumidor* de la computación paralela.

3 Marco teórico

3.1 Arquitectura de microservicios

Definición

Una **arquitectura de microservicios** organiza una aplicación como un conjunto de servicios pequeños e independientes, cada uno con una responsabilidad única, su propio proceso y su propia interfaz de comunicación. Los servicios se despliegan y escalan por separado y colaboran mediante mensajes —habitualmente HTTP/REST, colas o eventos.

Frente al modelo monolítico, los microservicios ofrecen **escalabilidad** (se puede replicar solo el servicio saturado), **mantenibilidad** (cada servicio se modifica sin recompilar el resto), **reuti-**

lización y tolerancia a fallos parcial (la caída de un servicio no necesariamente tumba todo el sistema).

Cuadro 1. Comparación entre el enfoque monolítico y el de microservicios.

Aspecto	Monolito	Microservicios
Escalado	Se replica toda la aplicación	Se replica solo el servicio saturado
Despliegue	Un único artefacto	Cada servicio se despliega por separado
Fallos	Un error puede tumbar todo	Aislamiento parcial de fallos
Acoplamiento	Alto entre módulos	Bajo; interfaces bien definidas

3.2 Comunicación HTTP/REST y serialización

Los servicios de esta práctica se comunican por **HTTP** intercambiando mensajes en formato **JSON**. Como JSON solo transporta texto, las imágenes (arreglos binarios de píxeles) se codifican en **Base64**: se comprimen a JPG/PNG en memoria y esos bytes se representan como una cadena de texto imprimible que viaja dentro del mensaje.

3.3 Patrón pipeline y patrón productor–consumidor

Pipeline (segmentación encauzada)

El **pipeline** divide una tarea en etapas secuenciales conectadas, donde la salida de una etapa es la entrada de la siguiente. Cuando hay varios elementos de trabajo, las etapas operan *simultáneamente* sobre elementos distintos: mientras la etapa 3 procesa el elemento i , la etapa 2 procesa el $i+1$ y la etapa 1 el $i+2$.

El pipeline es una forma de paralelismo *temporal*: no acelera una sola imagen, pero aumenta el *throughput* (imágenes por segundo) cuando llegan muchas. En el patrón **productor–consumidor**, uno o más productores generan trabajo y lo entregan a consumidores que lo procesan, desacoplando la velocidad de generación de la de consumo. En nuestro sistema, el **cliente** es el productor de imágenes y los microservicios son los consumidores.

3.4 Detección de rostros: Haar (Viola–Jones) + NMS

El análisis usa el detector **Haar** de OpenCV, un clasificador en cascada entrenado con Ada-Boost que evalúa *características rectangulares* (diferencias de intensidad entre regiones claras y oscuras del rostro, como los ojos frente a las mejillas). La Figura 1 muestra los cuatro tipos básicos de estas características.

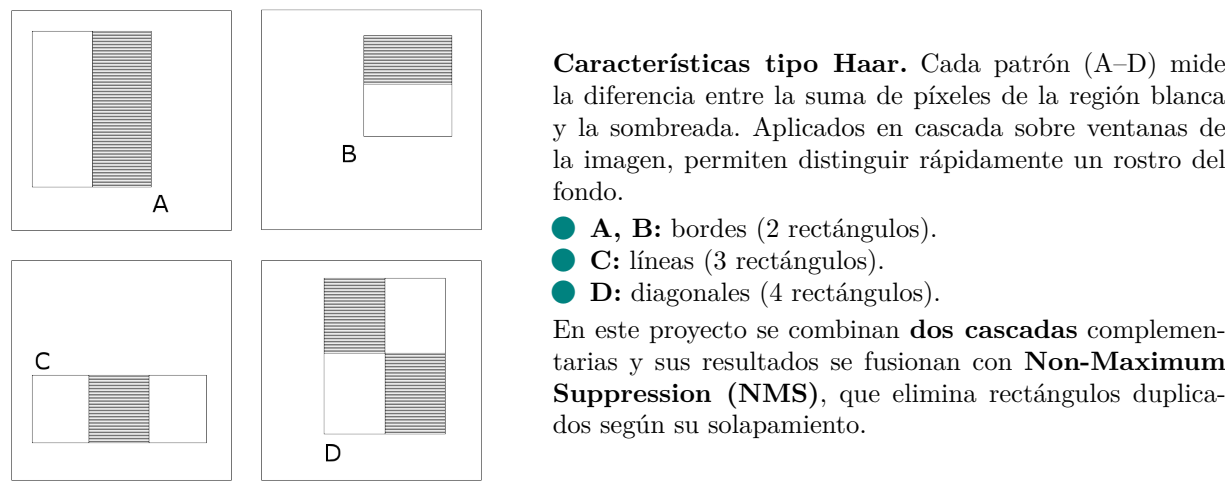


Figura 1. Características rectangulares tipo Haar empleadas por el detector Viola–Jones. Fuente: Wikimedia Commons (dominio público).

4 Actividad 1: Análisis del problema

4.1 Flujo general de procesamiento

El procesamiento de una imagen sigue cuatro pasos naturales, que se convierten en las cuatro etapas del sistema (Figura 2).

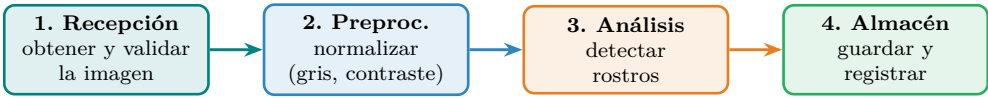


Figura 2. Flujo general del procesamiento de imágenes, convertido en las cuatro etapas del pipeline distribuido.

4.2 Tareas independientes y tareas que se comunican

Aspecto	Descripción
Tareas independientes	Cada etapa es autónoma: se desarrolla, se ejecuta y se escala por separado. El preprocesamiento no necesita saber cómo se detectan los rostros; solo entrega una imagen normalizada. Además, <i>imágenes distintas</i> pueden procesarse de forma independiente y simultánea (no comparten estado).
Tareas que se comunican	Las etapas están encadenadas: la salida de una es la entrada de la siguiente. Existe una dependencia de datos recepción → preprocesamiento → análisis → almacenamiento que obliga a comunicar el resultado parcial (la imagen y sus metadatos).

Observación clave para el paralelismo

Para *una misma* imagen las etapas son secuenciales (hay dependencia de datos), pero para *muchas* imágenes las etapas pueden solaparse en el tiempo. Ahí está la oportunidad de paralelismo que explota el pipeline.

5 Actividad 2: Diseño de la arquitectura

5.1 Diagrama de microservicios

La Figura 3 muestra la topología del sistema. El cliente (productor) envía la imagen al Microservicio 1 y cada servicio reenvía el trabajo al siguiente por HTTP; la respuesta final regresa por la misma cadena.

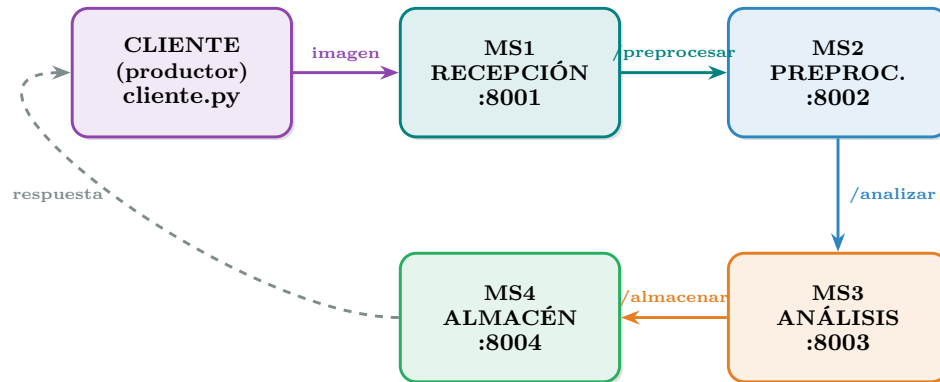


Figura 3. Arquitectura de microservicios en pipeline. Las flechas sólidas son peticiones HTTP hacia adelante; la flecha punteada es la respuesta que “burbujea” de vuelta hasta el cliente.

5.2 Entradas, salidas y responsabilidades

Microservicio	Puerto	Responsabilidad / Entrada → Salida
MS1 Recepción	8001	Puerta de entrada (gateway). Recibe la imagen (multipart), le asigna un ID único, guarda una copia cruda y construye el “sobre” JSON. <i>Entrada:</i> archivo de imagen. <i>Salida:</i> sobre JSON hacia MS2.
MS2 Preprocesamiento	8002	Normalización. Convierte a escala de grises, equaliza el histograma y aplica un filtro de reducción de ruido. <i>Entrada:</i> sobre con imagen. <i>Salida:</i> sobre con imagen en gris hacia MS3.
MS3 Análisis	8003	Detección de rostros. Aplica Haar (2 cascadas) + NMS y dibuja los recuadros sobre la imagen a color. <i>Entrada:</i> sobre con imagen preprocesada. <i>Salida:</i> sobre con rostros y coordenadas hacia MS4.
MS4 Almacenamiento	8004	Persistencia. Guarda la imagen anotada en disco y anexa el registro a la bitácora JSON/CSV con los tiempos por etapa. <i>Entrada:</i> sobre con imagen anotada. <i>Salida:</i> resumen JSON al cliente.

5.3 El “sobre” de datos (envelope)

La información viaja entre servicios dentro de un objeto JSON que cada etapa enriquece:

```

1 sobre = {
2   "id": "a2c11477",           # identificador unico de la imagen
3   "nombre": "persona2_michelle.jpg",
4   "imagen_b64": "...",       # imagen codificada en Base64
5   "meta": {"ancho": 520, "alto": 520, ...},
6   "rostros": 1,               # lo agrega MS3
7   "coordenadas": [[x, y, w, h], ...],
8   "etapas": [                 # marca de tiempo que agrega cada servicio
9     {"servicio": "RECEPCION", "inicio": ..., "fin": ...},
10    {"servicio": "PREPROC", "inicio": ..., "fin": ...}, ...
11  ]
12 }
  
```

6 Actividad 3: Implementación de microservicios

El sistema se implementó en **Python 3** con **Flask** (servidor HTTP), **OpenCV** (visión por computadora), **NumPy** y **requests**. Cada microservicio es un archivo independiente que se ejecuta en su propio proceso y puerto, y expone un endpoint `/health` para verificar que está activo.

6.1 Utilidades compartidas (comun.py)

Contiene la serialización Base64↔imagen, el detector Haar+NMS y una función de comunicación resiliente. Un detalle importante de **seguridad de hilos**: como el servidor atiende varias imágenes a la vez, cada hilo usa *su propio* clasificador Haar (almacenamiento *thread-local*), porque el objeto `CascadeClassifier` de OpenCV no es seguro para uso concurrente.

```

1 def imagen_a_base64(img, formato=".jpg"):
2     ok, buffer = cv2.imencode(formato, img)          # comprime en memoria
3     return base64.b64encode(buffer).decode("utf-8")
4
5 _local = threading.local()    # cada hilo tendra su propia pareja de cascadas
6 def _cascadas_del_hilo():
7     if not hasattr(_local, "cascadas"):
8         _local.cascadas = tuple(cv2.CascadeClassifier(a) for a in
9                                 _ARCHIVOS_CASCADAS)
10    return _local.cascadas

```

6.2 MS1 – Recepción

Actúa como gateway: valida la imagen, le asigna un ID, guarda una copia y la inyecta al pipeline.

```

1 @app.route("/procesar", methods=["POST"])
2 def procesar():
3     archivo = request.files["imagen"]
4     img = cv2.imdecode(np.frombuffer(archivo.read(), "uint8"), cv2.
5                       IMREAD_COLOR)
6     id_img = uuid.uuid4().hex[:8]
7     guardar_imagen(os.path.join(config.DIR_RECIBIDAS, f"{id_img}_{archivo.
8     filename}"), img)
9     sobre = {"id": id_img, "nombre": archivo.filename,
10             "imagen_b64": imagen_a_base64(img), "meta": {...}, "etapas":
11             [...]}
12     resp = post_resiliente(config.EP_PREPROCESAR, sobre)    # -> MS2
13     return jsonify(resp.json())

```

6.3 MS2 – Preprocesamiento

Normaliza la imagen con tres operaciones de OpenCV encadenadas.

```

1 gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)    # a) escala de grises
2 gris = cv2.equalizeHist(gris)                    # b) ecualizacion de
3                                                  histograma
4 gris = cv2.GaussianBlur(gris, (3, 3), 0)        # c) reduccion de ruido
5 sobre["gris_b64"] = imagen_a_base64(gris, ".png")
6 resp = post_resiliente(config.EP_ANALIZAR, sobre)    # -> MS3

```

6.4 MS3 – Análisis (detección de rostros)

Detecta los rostros sobre la imagen en gris y los dibuja sobre la imagen original a color.

```

1 gris = base64_a_imagen(sobre["gris_b64"]); original = base64_a_imagen(sobre["
  imagen_b64"])
2 rostros = detectar_rostros(gris) # Haar x2 + NMS
3 dibujar_rostros(original, rostros)
4 sobre["imagen_b64"] = imagen_a_base64(original) # imagen anotada
5 sobre["rostros"] = int(len(rostros))
6 resp = post_resiliente(config.EP_ALMACENAR, sobre) # -> MS4

```

6.5 MS4 – Almacenamiento

Persiste la imagen anotada y la bitácora; calcula la duración de cada etapa. Usa un candado (`threading.Lock`) para que las escrituras concurrentes a la bitácora sean seguras.

```

1 guardar_imagen(salida, base64_a_imagen(sobre["imagen_b64"]))
2 duraciones = {e["servicio"]: (e["fin"]-e["inicio"])*1000 for e in sobre["
  etapas"]}
3 with _lock: # escritura segura ante concurrencia
4     _anexar_json(registro); _anexar_csv(registro)
5 return jsonify(registro) # el resumen regresa al cliente por la cadena

```

7 Actividad 4: Comunicación distribuida

7.1 Mecanismo de intercambio

La comunicación es **HTTP síncrono** con cuerpo **JSON**. Cada servicio actúa como *cliente* del siguiente: usa `requests.post()` para enviar el sobre y espera la respuesta. La imagen viaja **codificada en Base64** dentro del JSON, lo que garantiza un formato de entrada/salida *consistente* entre todas las etapas.

7.2 Validación del paso de datos

Para comprobar que la imagen y el resultado pasan correctamente de una etapa a otra se usaron tres mecanismos:

- **Identificador único** (id) que acompaña a la imagen por todo el pipeline y aparece en los logs de los cuatro servicios.
- **Copia cruda** en recibidas/ e **imagen anotada** en procesadas/: comparar ambas confirma que el análisis operó sobre la imagen correcta.
- **Endpoints /health**: el cliente verifica que los cuatro servicios respondan antes de enviar nada.

7.3 Tolerancia a fallos: reintentos con backoff

En un sistema distribuido las peticiones pueden fallar de forma transitoria (una etapa está ocupada, la conexión se reinicia bajo carga). En lugar de abortar al primer error, las llamadas entre servicios se hacen con **reintentos y espera creciente**:

```

1 def post_resiliente(url, sobre, timeout=30, intentos=4, espera=0.15):
2     for intento in range(1, intentos + 1):
3         try:
4             resp = _sesion.post(url, json=sobre, timeout=timeout)
5             resp.raise_for_status(); return resp
6         except requests.RequestException as e:

```



```
7         if intento < intentos: time.sleep(espera * intento) # backoff
8         raise e
```

8 Actividad 5: Pruebas del sistema

Se procesó un lote de **6 imágenes** descargadas de repositorios de dominio público: cuatro retratos individuales, una fotografía grupal con muchas personas (prueba de estrés) y una imagen de control sin rostros. El sistema detectó correctamente **34 rostros** en total, incluyendo **0 rostros** en la imagen de control, lo que confirma que no genera falsos positivos evidentes en escenas sin personas.

8.1 Evidencia de imágenes procesadas

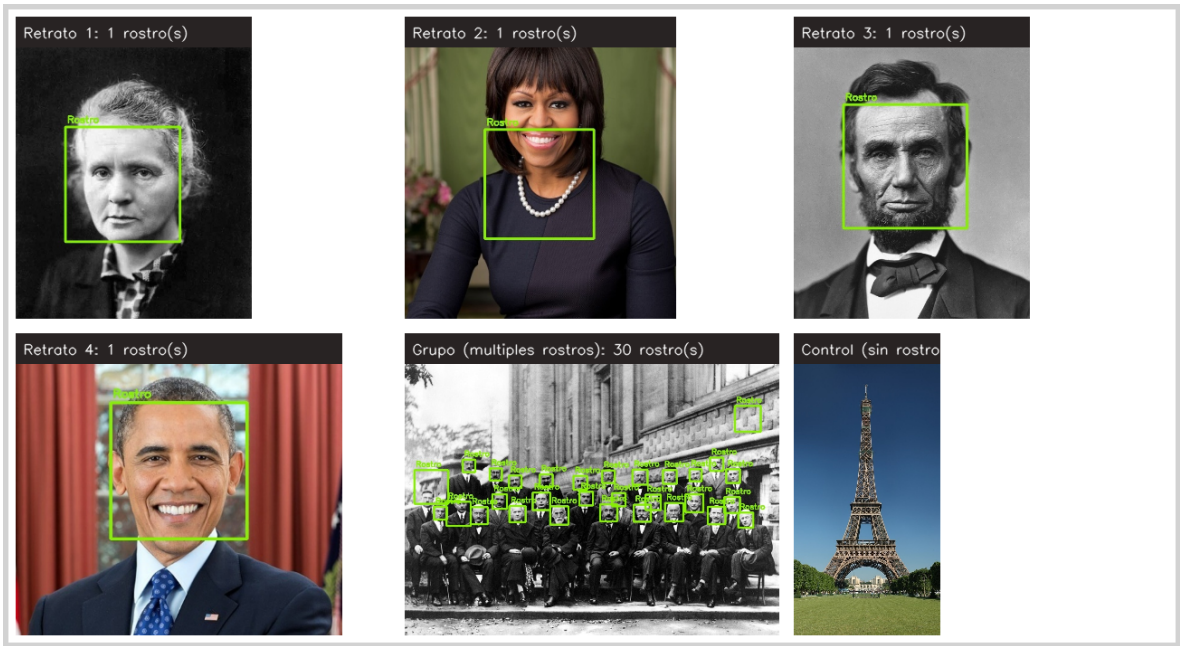


Figura 4. Imágenes procesadas por el pipeline. Cada rostro detectado se marca con un recuadro verde. Los cuatro retratos devuelven 1 rostro, la fotografía grupal 30 y la imagen de control (Torre Eiffel) devuelve 0 rostros, como se espera.



(a) Detección múltiple: 30 rostros en la fotografía grupal. (b) Detección de un rostro en un retrato individual.

Figura 5. Detalle de dos imágenes anotadas por el Microservicio de Análisis.

8.2 Tiempos por etapa

La Tabla 2 muestra la duración de cada etapa (modo secuencial). La etapa de **análisis** domina el costo, confirmándose como el *cuello de botella* del sistema. La **primera** imagen procesada es mucho más lenta por el *warm-up* (carga inicial de las cascadas Haar en el hilo).

Cuadro 2. Duración por etapa en milisegundos (una corrida secuencial).

Imagen	Recep.	Preproc.	Análisis	Almacén	Total	Rostros
control_sin_rostro [†]	17.8	32.0	1038.5	10.9	1126.5	0
grupo_conferencia	21.8	35.6	569.9	17.4	703.4	30
persona1_curie	9.8	14.3	333.8	6.7	392.0	1
persona2_michelle	10.5	16.9	280.0	17.1	362.9	1
persona4_lincoln	9.9	15.0	196.5	22.1	277.7	1
persona5_obama	8.7	16.7	262.1	17.9	347.5	1

[†] Primera imagen del lote: su análisis incluye el *warm-up* de cargar las cascadas Haar (~1 s).

8.3 Secuencial vs. concurrente

Se comparó el envío de las 6 imágenes en dos modos: **secuencial** (una a la vez) y **concurrente** (4 envíos simultáneos, de modo que varias imágenes avanzan a la vez por el pipeline). Los resultados se resumen en la Tabla 3 y la Figura 6.

Cuadro 3. Comparación de rendimiento (6 imágenes).

Métrica	Valor
Tiempo secuencial	3.498 s
Tiempo concurrente	1.416 s
Speedup	2.47×
Mejora	59.5 %
Rostros (total)	34

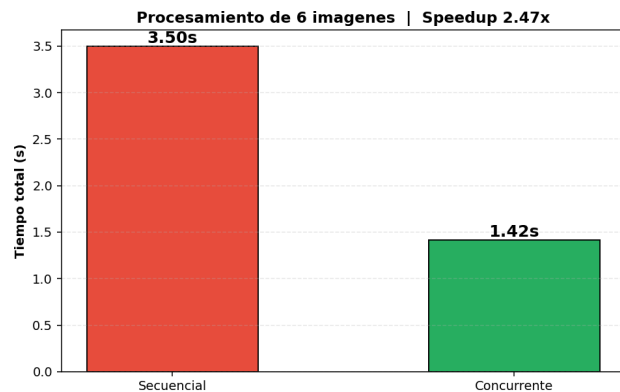


Figura 6. Tiempo total: modo secuencial frente a concurrente sobre la misma arquitectura.

8.4 Secuencia de ejecución observada

En los logs de los servicios se observó el comportamiento esperado del pipeline: en modo concurrente, el Microservicio 1 registra varias imágenes “recibidas” *antes* de que las primeras terminen, es decir, hay varias imágenes en distintas etapas al mismo tiempo. No se registraron errores tras incorporar los reintentos con backoff.

9 Actividad 6: Análisis desde la computación paralela

9.1 ¿Qué partes se ejecutan en paralelo?

La Figura 7 ilustra la diferencia entre ambos modos: en el secuencial las imágenes no se solapan, mientras que en el concurrente varias avanzan a la vez por etapas distintas del pipeline.

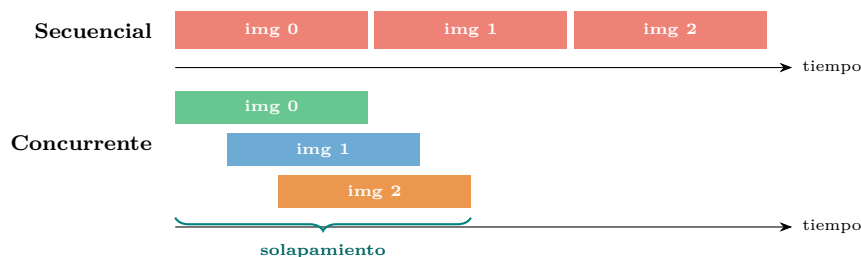


Figura 7. Paralelismo temporal del pipeline. En el modo concurrente las imágenes se solapan en etapas distintas, reduciendo el tiempo total del lote.

- **Entre etapas (pipeline):** las cuatro etapas son procesos distintos; cuando hay varias imágenes en vuelo, la recepción de una, el preprocesamiento de otra, el análisis de una tercera y el almacenamiento de una cuarta ocurren *simultáneamente*.
- **Dentro de una etapa (replicación):** cada servicio Flask corre con `threaded=True` y atiende varias peticiones a la vez; el análisis, al ser el cuello de botella, es el mejor candidato a replicarse.
- **Lo que NO se paraleliza:** el procesamiento de *una* imagen concreta es secuencial por su dependencia de datos (no se puede analizar antes de preprocesar).

9.2 Cuello de botella y ley de Amdahl

El análisis concentra más del 90 % del tiempo (Tabla 2). En un pipeline, el *throughput* lo fija la etapa más lenta; por tanto, la vía de mejora más efectiva es **escalar horizontalmente** el MS3 (varias instancias detrás de un balanceador). El speedup observado ($2.47\times$) es menor que el ideal ($4\times$) por el desbalance entre etapas, el *warm-up* inicial, el costo de serializar en Base64 y el límite del servidor de desarrollo de Flask —coherente con la ley de Amdahl y la sobrecarga propia de un sistema distribuido.

9.3 Cómo mejora la arquitectura la organización y escalabilidad

- **Organización:** cada tarea vive en un servicio con una única responsabilidad, lo que facilita entender, probar y mantener el sistema.
- **Escalabilidad:** se puede replicar solo el servicio saturado sin tocar los demás; el sistema crece por partes según la demanda.
- **Despliegue independiente:** cada microservicio se puede actualizar o desplegar (incluso en máquinas distintas o en contenedores) sin detener el resto.

10 Resultados obtenidos

- Una **arquitectura distribuida** de 4 microservicios claramente definida (Figura 3).
- Cuatro **microservicios funcionales** implementados y comunicados por HTTP.
- **Evidencia de comunicación** correcta: el mismo ID recorre los cuatro servicios y la imagen anotada corresponde a la imagen recibida.
- **Imágenes procesadas y resultados almacenados** en `procesadas/` y en la bitácora `registro.json / registro.csv`.
- **Análisis técnico:** speedup de $2.47\times$ y 59.5 % de mejora del modo concurrente frente al secuencial, con el análisis como cuello de botella.

11 Conclusiones

- Una solución distribuida con microservicios permite dividir el procesamiento de imágenes en componentes independientes, lo que facilita la escalabilidad, el mantenimiento y la reutilización del sistema.

- El sistema materializa dos patrones fundamentales de la computación paralela: el *pipeline* (paralelismo temporal entre etapas) y el *productor-consumidor* (desacople entre quien genera y quien procesa el trabajo).
- El procesamiento concurrente de varias imágenes redujo el tiempo total a menos de la mitad (speedup $2.47\times$), demostrando empíricamente el beneficio del paralelismo, aunque limitado por el cuello de botella del análisis y la sobrecarga de comunicación.
- La comunicación por HTTP con formatos de entrada/salida consistentes (JSON + Base64) y mecanismos de tolerancia a fallos (reintentos) resultó suficiente y robusta para coordinar las etapas del sistema.

12 Recomendaciones

- Diseñar cada microservicio con una **responsabilidad bien definida** y una interfaz clara.
- Mantener **formatos de entrada y salida consistentes** entre servicios (aquí, el “sobre” JSON con la imagen en Base64).
- **Probar cada microservicio por separado** (con su endpoint `/health` y peticiones individuales) antes de integrar el flujo completo.
- Para producción, sustituir el servidor de desarrollo de Flask por un servidor WSGI (Gunicorn/Uvicorn), añadir un **balanceador** frente al servicio de análisis y considerar una **cola de mensajes** para desacoplar aún más las etapas.

El manual de instalación y uso del sistema se entrega como documento independiente:
Manual_Usuario_Instalacion.pdf.